P1.T09.01.01. Структурни тип: Пример: Разломак

1. Структурни тип

Структуре су још један врло значајан тип који корисник (програмер) сам дефинише.

Оне представљају начин да од више података, који у датом контексту чине смисаону целину, направимо један „групни” податак.

Када желимо да неколико података третирамо као целину, након дефинисања неколико појединачних променљивих, податке можемо да групишемо у структуру (или торку) и доделимо једној променљивој.

На пример, опис једног такмичара чине подаци о његовом имену, разреду, школи из које долази и броју освојених поена.

Груписањем података у структуру наглашавамо и на нивоу писања кода смисаону повезаност података и тиме програм чинимо јаснијим. Осим тога, поједностављујемо прослеђивање такве групе података као параметра методу, враћање групе података као резултата рада метода и додељивање групе података другој групи.

Поља структуре

Податке од којих се структура састоји зовемо **поља структуре**.

Поља могу, али не морају да буду међусобно истог типа.

Такође, поља могу да буду и сложеног типа, укључујући друге структуре, низове итд.

У свом најпростијем облику, **структурни тип** се дефинише писањем кључне речи `struct`, иза које следи име типа, а затим у витичастим заградама списак поља.
На пример, структура **Knjiga**, која описује једну књигу, може да буде задата овако:

C++

```
#include <iostream>
using namespace std;
struct Knjiga
{
    string autor;
    string naslov;
    string oblast;
    int brStrana;
};
int main()
{
    return 0;
}
```

C#

```
using System;
class R1_T09_01_Strukture_Knjiga
{
    struct Knjiga
    {
        public string autor;
        public string naslov;
        public string oblast;
        public int brStrana;
    }
    static void Main()
    {
    }
}
```

Променљиве чији је тип нека структура такође зовемо структуре.
Када је потребно да разликујемо тип и променљиву, користимо термине:

- структурни тип и
- структурна променљива.

Приступање пољима структурне променљиве

У програму се пољима структуре приступа помоћу имена структурне променљиве и тачке. На пример, након дате дефиниције структуре **Knjiga**, њена поља **autor** и **brStrana** би могла да се употребе овако:

C++

```
#include <iostream>
using namespace std;

struct Knjiga
{
    string autor;
    string naslov;
    string oblast;
    int brStrana;
};
int main()
{
    Knjiga k;
    k.autor = "Bora Stankovic";
    k.brStrana = 806;
    k.brStrana++;
    cout << k.brStrana;
    return 0;
}
```

C#

```
using System;
class R1_T09_01_Strukture_Knjiga
{
    struct Knjiga
    {
        public string autor;
        public string naslov;
        public string oblast;
        public int brStrana;
    }
    static void Main()
    {
        Knjiga k;
        k.autor = "Bora Stankovic";
        k.brStrana = 806;
        k.brStrana++;
        Console.WriteLine(k.brStrana);
    }
}
```

Структуре нису референцирани, него вредносни тип података

Напоменимо још да, за разлику од низова, листи, стрингова и матрица, структуре нису референцирани, него **вредносни тип података**. По томе су структуре сличније простим типовима као што су целобројни и реални типови. То значи да се приликом додељивања једне структурне променљиве другој врши копирање садржаја свих поља. На пример, извршавањем (иначе бесмисленог) кода

C++

```
#include <iostream>
using namespace std;

struct Knjiga
{
    string autor;
    string naslov;
    string oblast;
    int brStrana;
};
int main()
{
        Knjiga k1, k2;
        k1.autor = "Bora Stankovic";
        k1.naslov = "Kostana";
        k1.oblast = "drama";
        k1.brStrana = 806;
        k2 = k1;
        k2.naslov = "Necista krv";
        k2.brStrana++;
        cout << k1.brStrana << k2.brStrana;
    return 0;
}
```

C#

```
using System;
class R1_T09_01_Strukture_Knjiga
{
    struct Knjiga
    {
        public string autor;
        public string naslov;
        public string oblast;
        public int brStrana;
    }
    static void Main()
    {
        Knjiga k1, k2;
        k1.autor = "Bora Stankovic";
        k1.naslov = "Kostana";
        k1.oblast = "drama";
        k1.brStrana = 806;
        k2 = k1;
        k2.naslov = "Necista krv";
        k2.brStrana++;
        Console.WriteLine(k1.brStrana);
        Console.WriteLine(k2.brStrana);
    }
}
```

била би исписана два различита броја, чиме само потврђујемо да свака променљива задржава свој простор у меморији, тј. не долази до коришћења истог простора као што је то случај код референцираних типова података.

Пример 1: Задатак: Разломци

Напиши програм који сабира и множи разломке.

Улаз

Са стандардног улаза се уносе два разломка, сваки у посебном реду. Сваки разломак је записан тако што је су записани његов бројилац и именилац, раздвојени карактером **/**.

Излаз

На стандардни излаз исписати у првом реду збир, а у другом производ разломака. Разломци се исписују у истом формату у ком су учитани, максимално скраћени (бројилац и именилац су узајамно прости).
Пример

Пример

Улаз

2/3
1/4

Излаз

11/12
1/6

Решење

Овај задатак се може решити без коришћења структурног типа, тако што ћемо користити 4 целобројне променљиве, односно за сваки разломак по две: по једну за бројилац и по једну за именилац, као на пример:

C++

```cpp
#include <iostream>
using namespace std;
int main()
{
    int brojilac1, imenilac1, brojilac2, imenilac2;
    return 0;
}
```

C#

```csharp
using System;
class R1_T09_01_Strukture_Razlomak
{
    static void Main()
    {
        int brojilac1, imenilac1, brojilac2, imenilac2;
    }
}
```

Корак 1: Груписање података: Дефинисање структурног типа (и поља) Разломак

Ипак, много је практичније користити структурни тип података. Сваки разломак има два податка, односно два поља, једно за бројилац и друго за именилац и оба поља су целобројног типа, па можемо да их групишемо у структурни тип података, који ћемо назвати, на пример `Razlomak`:

C++

```cpp
#include <iostream>
using namespace std;
struct Razlomak
{
    int brojilac, imenilac;
};
int main()
{
    return 0;
}
```

C#

```csharp
using System;
class R1_T09_01_Strukture_Razlomak
{
    struct Razlomak
    {
        public int brojilac, imenilac;
    }
    static void Main()
    {
    }
}
```

Корак 2: Учитавање (унос) улазних вредности структурне променљиве за оба разломка

Учитавање улазних вредности и додељивање пољима структурним промененљивама типа `Razlomak` можемо реализовати у главном делу програма, с тим што је потребно да водимо рачуна о томе да су за сваки разломак обе улазне вредности бројева раздвојени карактером **/**:

C++

```
#include <iostream>
using namespace std;

struct Razlomak
{
    int brojilac, imenilac;
};

int main()
{
    char c;
    Razlomak r1, r2;
    cin >> r1.brojilac >> c >> r1.imenilac;
    cin >> r2.brojilac >> c >> r2.imenilac;
    return 0;
}
```

C#

```
using System;
class R1_T09_01_Strukture_Razlomak
{
    struct Razlomak
    {
        public int brojilac, imenilac;
    }
    static void Main()
    {
        Razlomak r1, r2;
        string[] s1 = Console.ReadLine().Split('/');
        r1.brojilac = int.Parse(s1[0]);
        r1.imenilac = int.Parse(s1[1]);
        string[] s2 = Console.ReadLine().Split('/');
        r2.brojilac = int.Parse(s2[0]);
        r2.imenilac = int.Parse(s2[1]);
    }
}
```

Практичније је да учитавање улазних вредности разломка издвојимо у посебну методу:

C++

```
#include <iostream>
using namespace std;
struct Razlomak
{
    int brojilac, imenilac;
};
Razlomak ucitaj()
{
    Razlomak r;
    char c;
    cin >> r.brojilac >> c >> r.imenilac;
    skrati(r);
    return r;
}
int main()
{
    Razlomak r1 = ucitaj();
    Razlomak r2 = ucitaj();
    return 0;
}
```

C#

```
using System;
class R1_T09_01_Strukture_Razlomak
{
    struct Razlomak
    {
        public int brojilac, imenilac;
    }
    static Razlomak ucitaj()
    {
        Razlomak r;
        string[] s = Console.ReadLine().Split('/');
        r.brojilac = int.Parse(s[0]);
        r.imenilac = int.Parse(s[1]);
        return r;
    }
    static void Main()
    {
        Razlomak r1 = ucitaj();
        Razlomak r2 = ucitaj();
    }
}
```

Пожељно је да тестирамо извршавање програмског кода у развојном окружењу за дате улазне податке из текста задатка, односно за разломке чије су вредности **2/3** и **1/4**.

Наравно, нећемо добити излазне вредности на екрану у прозору конзолне апликације, јер још увек нисмо написали наредбе излаза, али ћемо проверити да ли наш програм исправно учитава улазне податке. Препорука је да програм извршавамо корак по корак и посматрамо тренутне вредности променљивих.

Корак 3: Приказивање (испис) вредности поља структурне променљиве за оба разломка

Иако још увек нисмо написали главни део програмског кода за израчунавање збира и разлике два разломка, препорука је да сада напишемо део кода који ће приказати (исписати) тренутне вредности оба поља сваког разломка.
Практичније је да исписивање вредности разломка издвојимо у посебну методу:

C++

```cpp
#include <iostream>
using namespace std;
struct Razlomak
{
    int brojilac, imenilac;
};
Razlomak ucitaj()
{
    Razlomak r;
    char c;
    cin >> r.brojilac >> c >> r.imenilac;
    return r;
}
void ispisi(const Razlomak &r)
{
    cout << r.brojilac << "/" << r.imenilac << endl;
}
int main()
{
    Razlomak r1 = ucitaj();
    Razlomak r2 = ucitaj();
    ispisi(r1);
    ispisi(r2);
    return 0;
}
```

C#

```cs
using System;
class R1_T09_01_Strukture_Razlomak
{
    struct Razlomak
    {
        public int brojilac, imenilac;
    }
    static Razlomak ucitaj()
    {
        Razlomak r;
        string[] s = Console.ReadLine().Split('/');
        r.brojilac = int.Parse(s[0]);
        r.imenilac = int.Parse(s[1]);
        return r;
    }
    static void ispisi(Razlomak r)
    {
        Console.WriteLine(r.brojilac+"/"+r.imenilac);
    }
    static void Main()
    {
        Razlomak r1 = ucitaj();
        Razlomak r2 = ucitaj();
        ispisi(r1);
        ispisi(r2);
    }
}
```

Пожељно је да тестирамо извршавање програмског кода у развојном окружењу за дате улазне податке из текста задатка, односно за разломке чије су вредности **2/3** и **1/4**.

Наравно, нећемо добити исправне излазне вредности на екрану у прозору конзолне апликације, јер још увек нисмо написали главни део програма за израчунавање збира и производа два разломка, али ћемо проверити да ли наш програм исправно учитава улазне податке и да ли их исправно (исте вредности које су и учитане) исписује.

Корак 4: Израчунавање збира и производа два разломка

Сада можемо да имплементирамо главни део задатка, односно да применимо познате формуле из математике за израчунавање збира и производа два разломка.
Практичније је да израчунавање вредности збира и производа два разломка издвојимо у посебне методе:

C++

```cpp
#include <iostream>
using namespace std;

struct Razlomak
{
    int brojilac, imenilac;
};
Razlomak ucitaj()
{
    Razlomak r;
    char c;
    cin >> r.brojilac >> c >> r.imenilac;
    return r;
}
void ispisi(const Razlomak &r)
{
    cout << r.brojilac << "/" << r.imenilac << endl;
}
Razlomak saberi(const Razlomak &a, const Razlomak &b)
{
    Razlomak r;
    r.brojilac = a.brojilac * b.imenilac +
                 a.imenilac * b.brojilac;
    r.imenilac = a.imenilac * b.imenilac;
    return r;
}
Razlomak mnozi(const Razlomak &a, const Razlomak &b)
{
    Razlomak r;
    r.brojilac = a.brojilac * b.brojilac;
    r.imenilac = a.imenilac * b.imenilac;
    return r;
}
int main()
{
    Razlomak r1 = ucitaj();
    Razlomak r2 = ucitaj();
    ispisi(saberi(r1, r2));
    ispisi(mnozi(r1, r2));
    return 0;
}
```

C#

```cs
using System;
class R1_T09_01_Strukture_Razlomak
{
    struct Razlomak
    {
        public int brojilac, imenilac;
    }
    static Razlomak ucitaj()
    {
        Razlomak r;
        string[] s = Console.ReadLine().Split('/');
        r.brojilac = int.Parse(s[0]);
        r.imenilac = int.Parse(s[1]);
        return r;
    }
    static void ispisi(Razlomak r)
    {
        Console.WriteLine(r.brojilac+"/"+r.imenilac);
    }
    static Razlomak saberi(Razlomak a, Razlomak b)
    {
        Razlomak r;
        r.brojilac = a.brojilac * b.imenilac +
                     a.imenilac * b.brojilac;
        r.imenilac = a.imenilac * b.imenilac;
        return r;
    }
    static Razlomak mnozi(Razlomak a, Razlomak b)
    {
        Razlomak r;
        r.brojilac = a.brojilac * b.brojilac;
        r.imenilac = a.imenilac * b.imenilac;
        return r;
    }
    static void Main()
    {
        Razlomak r1 = ucitaj();
        Razlomak r2 = ucitaj();
        ispisi(saberi(r1, r2));
        ispisi(mnozi(r1, r2));
    }
}
```

Пожељно је да тестирамо извршавање програмског кода у развојном окружењу за дате улазне податке из текста задатка, односно за разломке чије су вредности **2/3** и **1/4**.

Наравно, нећемо добити потпуно коректне излазне вредности на екрану у прозору конзолне апликације, јер још увек нисмо написали у главном делу програма за скраћивање разломака, након израчунавања њиховог збира и производа, али ћемо проверити да ли наш програм исправно учитава улазне податке и да ли их исписује очекиване (израчунате, без скраћивања) излазне податке.

Корак 5: Скраћивање разломка

На крају, потребно је да допишемо наредбе којима ћемо максимално скратити вредности разломка, користећи уобичајена правила из математике. Да би скратили разломак потребно је да израчунамо највећи заједнички делилац (НЗД) за именилац и бројилац. Пожељено је да напишемо методу (функцију) за израчунавање најмањег заједничког делиоца (НЗД) два природна броја, коришћењем Еуклидовог алгоритма. Препорука је да се мало подсетимо **Еуклидовог алгоритма**.

Коначна верзија програмског кода, односно решења задатка би била попут следећег:

C++

```
#include <iostream>
using namespace std;

struct Razlomak
{
    int brojilac, imenilac;
};
Razlomak ucitaj()
{
    Razlomak r;
    char c;
    cin >> r.brojilac >> c >> r.imenilac;
    skrati(r);
    return r;
}
void ispisi(const Razlomak &r)
{
    cout << r.brojilac << "/" << r.imenilac << endl;
}
Razlomak saberi(const Razlomak &a, const Razlomak &b)
{
    Razlomak r;
    r.brojilac = a.brojilac * b.imenilac +
                 a.imenilac * b.brojilac;
    r.imenilac = a.imenilac * b.imenilac;
    skrati(r);
    return r;
}
Razlomak mnozi(const Razlomak &a, const Razlomak &b)
{
    Razlomak r;
    r.brojilac = a.brojilac * b.brojilac;
    r.imenilac = a.imenilac * b.imenilac;
    skrati(r);
    return r;
}
int NZD(int a, int b)
{
    while (b != 0)
    {
        int p = a % b;
        a = b;
        b = p;
    }
    return a;
}
void skrati(Razlomak &r)
{
    int nzd = NZD(r.brojilac, r.imenilac);
    r.brojilac /= nzd;
    r.imenilac /= nzd;
}
int main()
{
    Razlomak r1 = ucitaj();
    Razlomak r2 = ucitaj();
    ispisi(saberi(r1, r2));
    ispisi(mnozi(r1, r2));
    return 0;
}
```

C#

```
using System;
class R1_T09_01_Strukture_Razlomak
{
    struct Razlomak
    {
        public int brojilac, imenilac;
    }
    static Razlomak ucitaj()
    {
        Razlomak r;
        string[] s = Console.ReadLine().Split('/');
        r.brojilac = int.Parse(s[0]);
        r.imenilac = int.Parse(s[1]);
        return r;
    }
    static void ispisi(Razlomak r)
    {
        Console.WriteLine(r.brojilac+"/"+r.imenilac);
    }
    static Razlomak saberi(Razlomak a, Razlomak b)
    {
        Razlomak r;
        r.brojilac = a.brojilac * b.imenilac +
                     a.imenilac * b.brojilac;
        r.imenilac = a.imenilac * b.imenilac;
        skrati(ref r);
        return r;
    }
    static Razlomak mnozi(Razlomak a, Razlomak b)
    {
        Razlomak r;
        r.brojilac = a.brojilac * b.brojilac;
        r.imenilac = a.imenilac * b.imenilac;
        skrati(ref r);
        return r;
    }
    static int NZD(int a, int b)
    {
        while (b != 0)
        {
            int p = a % b;
            a = b;
            b = p;
        }
        return a;
    }
    static void skrati(ref Razlomak r)
    {
        int nzd = NZD(r.brojilac, r.imenilac);
        r.brojilac /= nzd;
        r.imenilac /= nzd;
    }
    static void Main()
    {
        Razlomak r1 = ucitaj();
        Razlomak r2 = ucitaj();
        ispisi(saberi(r1, r2));
        ispisi(mnozi(r1, r2));
    }
}
```

Пожељно је да тестирамо извршавање програмског кода у развојном окружењу за дате улазне податке из текста задатка, односно за разломке чије су вредности **2/3** и **1/4**, али и за још неке тест примере, које ћемо сами припремити, као што су, на пример:

1/2 и **1/4**, **1/2** и **1/2**, **3/2** и **2/4**, **1/1** и **2/1**, **0/1** и **0/2**, **-1/5** и **2/-1**, **7/5** и **4/3** и сл.

2. Укратко (уместо закључка)

Као и сваки други задатак и овај можемо решити на неколико тачних начина, као на пример:

1. Коришћењем само **основних типова података** (`int`) за 4 улазне променљиве.
2. Коришћењем **структурног** (`struct`) типа података за разломак, као у овој лекцији.
3. Коришћењем типа података за **уређене торке** (`Tuple`), што ће бити обијашњено у наредној лекцији.
4. Коришћењем других типова података, као на пример **колекције** (низ, листа).

Литература

Детаљније (извори):

- **Курс** на Петљи за **1. разред** гимназијског одељења: **Тема 9.1. Структуре**.
- **Збирка** на Петљи: **Методичка збирка задатака из основа програмирања**: **Структурни тип**.
- **Методичка збирка задатака из основа програмирања**: Задатак: **Разломци**.
- Википедија: **Еуклидов алгоритам**.
- Обавезно пажљиво прочитати лекцију **Еуклидов алгоритам** у оквиру **теме 5.5 Дељивост** на **курсу на Петљи за 1. разред**.
- Препорука је и да се пажљиво прочита лекција **Еуклидов алгоритам**.
- **Збирка** на Петљи: **Методичка збирка задатака из основа програмирања**: **Торке**.
- GitHub: Дељивост: НЗД и Еуклидов алгоритам.
- GitHub: Структурни тип: Разломак.
- GitHub: **Програмски код у C#**.
- GitHub: **Програмски код у C++**.
- Арена: Задатак: Разломци.
- Арена: Структурни тип: Задаци за вежбање: **R2_T09_Strukture_21**.